

CSCE 628 COMPUTATIONAL BIOLOGY · MODULE 04 · LECTURE NOTES

Search and Indexing

When quadratic is too slow, and what you give up.

MODULE OUTCOMES

- Explain seed-and-extend heuristics.
- Explain what BLAST trades for its speed.
- Explain suffix structures and their space cost.
- Explain the BWT and the FM-index.
- Choose a search strategy from the requirement.

1 Seed and extend

THE PROBLEM

Smith-Waterman against a 40-billion-base database is $O(\text{query} \times \text{database})$. Far too slow for interactive use.

THE HEURISTIC

- 1 break the query into short words (seeds)
- 2 find exact (or near-exact) word matches in a precomputed index of the database
- 3 extend each hit outward without gaps while the score keeps improving
- 4 keep the good ones, and do a full gapped alignment only on those

THE ASSUMPTION

a real alignment contains at least one exact word match of length w .

WHICH IS USUALLY TRUE AND NOT ALWAYS. Raising w makes the search faster and less sensitive; lowering w does the reverse. That dial IS the trade.

The word length is the sensitivity dial, and the assumption — that a true alignment contains an exact word match — is what can fail. Every fast search method in this course has an assumption of this shape: a cheap filter that is applied first, and that is believed not to discard anything important. Naming the filter is how you reason about what a tool can miss.

SO A HEURISTIC SEARCH CAN MISS TRUE ALIGNMENTS, AND THE MISS RATE IS RARELY REPORTED

BLAST is not guaranteed to find the optimal local alignment — it finds alignments that contain a seed, which is a different set from the set of high-scoring alignments, overlapping it heavily but not containing it.

And remote homologues are exactly the cases most likely to be missed, because they are by definition the ones least likely to contain a long exact match — **so the misses are not random, they are concentrated precisely where the interesting biology is.**

Which is why profile methods exist (Module 07 §3) — a profile uses a whole family's position-specific conservation pattern rather than a single sequence, and finds relationships that pairwise search cannot, at the cost of needing the family first.

So 'no significant hits' means 'this tool, with these parameters, against this database version, found none' — which is a very much weaker statement than it is usually taken to be, and is Module 13's point in miniature.

2 Suffix structures

Structure	Query time	Space
Suffix tree	$O(m)$ for a pattern of length m	Roughly 20 bytes per base with pointers — which for a human genome is tens of gigabytes. Too much.
Suffix array	$O(m \log n)$, or $O(m)$ with extra tables	Four to eight bytes per base — better, and still large.
FM-index (built on the BWT)	$O(m)$, by backward search (§3)	Under one byte per base — the index is smaller than the text.
k-mer hash table	$O(1)$ per word	Large, and the word length is fixed at build time — which is what §1's tools use.

The FM-index is why read mapping onto a human genome fits in a few gigabytes — the space reduction is what made Module 05 practical on ordinary hardware, and it arrived at roughly the time short-read sequencing did, which is a fortunate coincidence. **Space, not time, is what decided which structure won:** suffix trees have been asymptotically optimal for query time since 1973 and are not what anybody uses.

3 The BWT

CONSTRUCTION

take all rotations of the string (plus a terminator character), sort them lexicographically, and take the last column. That column is the Burrows-Wheeler transform.

WHY IT IS USEFUL

identical characters cluster together, so it compresses well -- which is why it was invented, for compression, and not for biology.

THE SEARCH PROPERTY (LF-mapping)

the i -th occurrence of character c in the last column corresponds to the i -th occurrence of c in the first column. That correspondence lets you walk backwards through the original text.

SO BACKWARD SEARCH

match the pattern right to left, maintaining the range of sorted rows prefixed by the matched suffix. $O(m)$, with a small index.

AND IT IS REVERSIBLE: the original string is recoverable from the transform, which is what makes it an index rather than a lossy summary.

The BWT was invented for compression and turned out to be searchable — which is why the index can be smaller than the text it indexes, a property that sounds impossible until you notice that the transform is the text, rearranged. Approximate matching is handled by backtracking the backward search: when a mismatch exhausts the range, try the other characters, which costs exponentially in the number of mismatches allowed and is why these tools permit only a few.

4 Choosing

- **Exact or approximate matching?** — exact matching is solved and cheap; approximate matching is where all the difficulty is, and every method handles mismatches either by backtracking (§3) or by seeding (§1).
- **One query or millions?** — an index pays off across many queries and is wasted on one — and Module 05's read mapping is emphatically the many case, with hundreds of millions of queries against a fixed reference.
- **Is the reference fixed?** — if so, build the index once and amortise it; if the database changes constantly, the indexing cost itself matters and a hash table may win.
- **How sensitive must the search be?** — which sets the seed length, and should be chosen from the expected divergence between query and target rather than from the tool's default, which was chosen for someone else's problem.

- **And what will you report when it finds nothing?** (§1's callout) — **which is the question this module most wants asked.** 'No significant hits' is a statement about a tool and its parameters, not about the biology — and it is reported as though it were the latter, routinely.

AND THE PATTERN TO CARRY FORWARD

Exact is quadratic, and everything faster assumes something (Module 02 §4's organising sentence) — and here the assumption is that a true alignment contains an exact seed of the chosen length.

And the assumption is usually reasonable — these tools work, they are extremely well tested, and the whole field depends on them — **which is exactly why it is so easy to stop noticing that it is an assumption** rather than a guarantee.

So the discipline is to name it when reporting a negative result — 'BLASTP with default parameters against this database found no significant hits' is checkable, and 'there is no homologue' is not, and the two are routinely written as though they were the same sentence.

Which is the program's rule applied to a search tool: state what you ran, and do not promote the tool's output into a biological claim (Module 13 §1's chain).

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
Altschul et al. — Basic local alignment search tool (free)	§1 in the original — and the heuristic's assumption is stated openly in it, which is worth seeing.
Burrows & Wheeler — A block-sorting lossless data compression algorithm	§3 in the original , where it is entirely about compression.
Ferragina & Manzini — Opportunistic data structures with applications	§3's FM-index — the search property, and the space bound that made it matter.
Langmead & Salzberg — Fast gapped-read alignment with Bowtie 2 (free)	§§3 and 4 applied — and the bridge to Module 05.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 **Implement seed-and-extend** with a hash of k-mers.
- 2 **Vary the word length** and measure speed against sensitivity.

- 3 **Construct a pair** that a given seed length provably misses.
- 4 **Compare BLAST's hits** to full Smith-Waterman on a small database.
- 5 **Build a suffix array** for a short string and search it.
- 6 **Compute a BWT by hand** for a ten-character string.
- 7 **Verify LF-mapping** on it.
- 8 **Implement backward search** and confirm it finds all occurrences.
- 9 **Measure index sizes** for the same text, three ways.
- 10 **Rewrite one 'there is no homologue' claim** as a checkable statement.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 Give the four steps of seed and extend.
- 2 What is the assumption, and what is the dial?
- 3 Why are the misses concentrated rather than random?
- 4 Why do profile methods help?
- 5 What does 'no significant hits' actually mean?
- 6 Name four index structures with their time and space.
- 7 Which won, and on what criterion?
- 8 Describe the BWT's construction and its search property.
- 9 Why is the index smaller than the text?
- 10 Give the five questions that choose a search strategy.